



DOCUMENTACIÓN TÉCNICA DE DATOS

Aplicación TRIP

Modelo de entidades · Fuentes de datos · Diccionario de datos

PixelPath

www.pixelpath.com

2026

Tabla de contenido

Tabla de contenido	2
Parte I · Modelo de entidades	4
Propósito	4
Diagrama entidad-relación	4
Descripción de entidades.....	5
Relaciones principales.....	5
Usuario y perfil.....	5
Perfil y partidas	5
Perfil y compras.....	5
Producto y compras	5
Perfil y progreso	5
Partidas y leaderboard.....	5
Reglas de negocio representadas	5
Uso analítico.....	6
Pendientes del modelo	6
Parte II · Fuentes de datos	7
Propósito	7
Resumen de fuentes	7
1. Supabase PostgreSQL.....	7
Descripción	7
Tablas y vistas utilizadas	7
Campos analíticos principales	7
Problemas o controles	8
2. Funciones SQL de simulación.....	8
Archivo	8
Descripción	8
Funciones.....	8
Clasificación.....	8
Limitaciones	8
3. Supabase Realtime.....	9
Descripción	9
Eventos utilizados	9
Controles	9
Limitaciones	9
4. Supabase Storage.....	9
Descripción.....	9

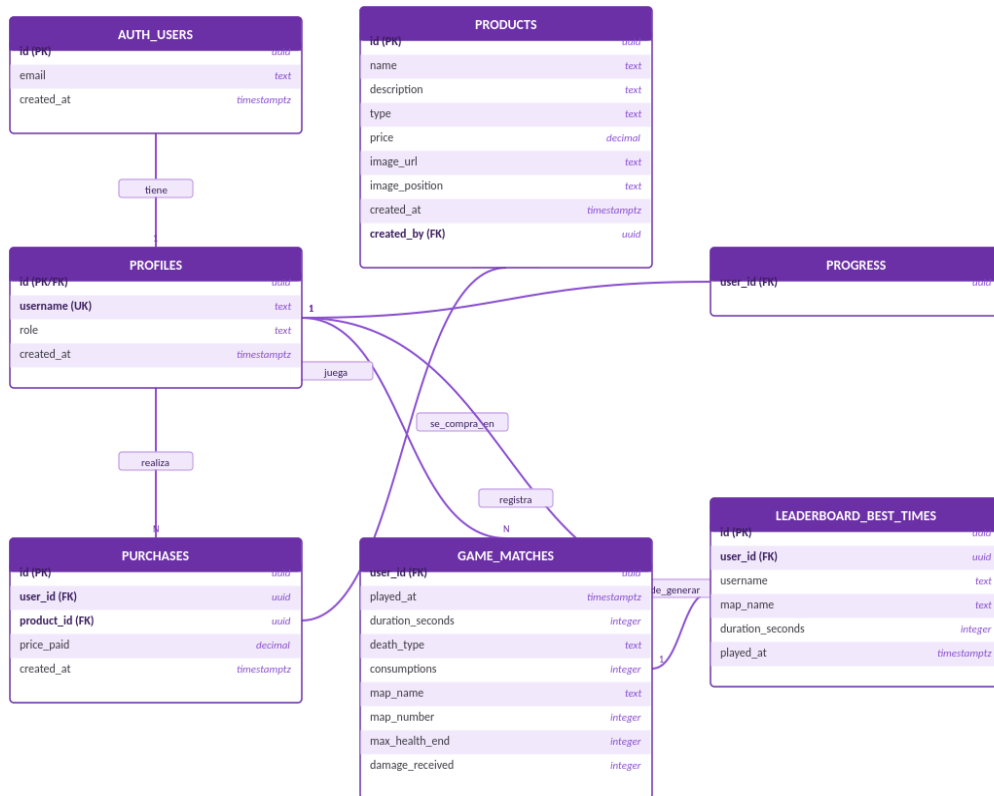
Clasificación.....	9
Controles	9
Registro de extracción recomendado	Error! Bookmark not defined.
Clasificación general	10
Reglas de privacidad	10
Parte III · Diccionario de datos.....	11
Propósito	11
Convenciones	11
profiles.....	11
products	11
purchases	12
game_matches	12
progress	12
leaderboard_best_times.....	13
Catálogos y dominios	13
Tipos de producto	13
Mapas simulados	13
Tipos de muerte simulados.....	13
Reglas de integridad	13
Pendientes de validación.....	14

Parte I · Modelo de entidades

Propósito

Este modelo representa las entidades principales del negocio digital TRIP y sus relaciones para analizar jugadores, partidas, progreso, productos y compras.

Diagrama entidad-relación



Nota: Las columnas marcadas en el diagrama se basan en las consultas y funciones disponibles. Las claves y campos exactos deben confirmarse en el esquema real de Supabase.

Descripción de entidades

Entidad	Función en el negocio	Grano del registro
auth.users	Identidad y autenticación	Una fila por cuenta
profiles	Perfil, nombre público y rol	Una fila por usuario
products	Catálogo de contenido digital	Una fila por producto
purchases	Transacciones de compra	Una fila por compra
game_matches	Actividad y resultado de una partida	Una fila por partida
progress	Avance persistente del jugador	Una fila o conjunto de filas por jugador, por confirmar
leaderboard_best_times	Récords mostrados en el ranking	Una fila por récord publicado

Relaciones principales

Usuario y perfil

auth.users.id se relaciona con profiles.id. El perfil contiene datos de aplicación como username, role y created_at, mientras que la cuenta autenticada conserva el correo y la identidad de acceso.

Perfil y partidas

Un perfil puede tener cero o muchas partidas. Cada partida pertenece a un solo user_id.

Perfil y compras

Un perfil puede realizar cero o muchas compras. Cada compra pertenece a un usuario y referencia un producto.

Producto y compras

Un producto puede no tener compras o tener muchas. El precio histórico de la operación se conserva en purchases.price_paid, por lo que no debe reemplazarse con el precio actual del catálogo al analizar ingresos históricos.

Perfil y progreso

Un perfil puede tener un registro de progreso. La estructura exacta de progress debe confirmarse en Supabase.

Partidas y leaderboard

Una partida puede producir un récord, pero no todas las partidas aparecen en el leaderboard. La vista o tabla leaderboard_best_times concentra los mejores tiempos publicados.

Reglas de negocio representadas

- Los administradores se excluyen de los KPI generales del negocio.
- Los jugadores pueden tener varias partidas y compras.
- Una compra debe utilizar un producto existente.

- Los productos mesh pueden comprarse varias veces; los demás tipos se manejan como compra única en la interfaz.
- Una partida no debe ocurrir antes del registro del jugador.
- La simulación genera bots separados de los usuarios reales.
- La eliminación y regeneración de bots no debe eliminar partidas ni compras de usuarios reales.
- El acceso a entidades privadas depende de las políticas RLS.

Uso analítico

Pregunta	Entidades y campos
¿Cuántos usuarios están activos?	profiles.id, game_matches.user_id, game_matches.played_at
¿Cuál es la retención?	profiles.created_at, game_matches.played_at, game_matches.user_id
¿Qué productos venden más?	purchases.product_id, purchases.price_paid, products.name
¿Cuál es el ingreso total?	purchases.price_paid, purchases.created_at
¿Qué mapas concentran actividad?	game_matches.map_name, game_matches.map_number
¿Qué jugadores compran?	profiles.id, purchases.user_id
¿Qué mapas presentan dificultad?	game_matches.map_name, death_type, duration_seconds, damage_received

Pendientes del modelo

- Confirmar claves primarias y foráneas mediante el esquema SQL base.
- Confirmar si leaderboard_best_times es una vista o una tabla.
- Documentar el detalle completo de progress.
- Añadir categorías, campañas, sesiones o dispositivos si se incorporan al modelo analítico.
- Crear una imagen exportada del diagrama para la presentación.

Parte II · Fuentes de datos

Propósito

Este documento identifica las fuentes que alimentan TRIP, su formato, clasificación, uso, problemas conocidos y controles de acceso.

Resumen de fuentes

Fuente	Tipo	Formato	Datos aportados	Uso analítico	Estado
Supabase PostgreSQL	Base de datos	Tablas relacionales	Perfiles, productos, compras, partidas y progreso	Fuente principal de KPI y dashboards	Implementada
Funciones SQL de simulación	Generador	PL/pgSQL	Bots, fechas, partidas y compras	Dataset histórico simulado	Implementada
Supabase Realtime	Flujo de eventos	Eventos PostgreSQL	Nuevas partidas y compras	Actualización de dashboards sin recarga	Parcial
Supabase Storage	Almacenamiento de archivos	Imágenes	Imágenes de productos	Presentación del catálogo	Implementada

Nota: La guía académica recomienda al menos tres fuentes. TRIP cuenta con una base persistente, un generador simulado y un flujo de eventos. Para una evidencia más sólida, se recomienda exportar una muestra CSV/JSON y documentarla como fuente derivada sin modificar manualmente los datos.

1. Supabase PostgreSQL

Descripción

Base de datos principal de la aplicación. Almacena la información persistente del negocio y permite que los dashboards consulten datos históricos.

Tablas y vistas utilizadas

`profiles`: usuarios de la aplicación, nombre y rol.

`products`: catálogo y precios.

`purchases`: transacciones y precio pagado.

`game_matches`: partidas, duración, mapas y resultados.

`progress`: avance del jugador.

`leaderboard_best_times`: mejores tiempos publicados.

`auth.users`: identidad administrada por Supabase Auth.

Campos analíticos principales

Fechas: `created_at`, `played_at`.

Identificadores: `id`, `user_id`, `product_id`.

Medidas: `price`, `price_paid`, `duration_seconds`, `consumptions`, `damage_received`.

Categorías: `role`, `type`, `death_type`, `map_name`.

Problemas o controles

- PostgREST puede devolver como máximo 1000 filas por consulta; el dashboard administrativo pagina las consultas.
- Se requiere RLS para limitar los datos de jugadores y proteger la administración.
- Los administradores se filtran de las métricas generales.
- Las fechas deben validarse para evitar registros futuros o compras anteriores al registro.

2. Funciones SQL de simulación

Archivo

```
supabase_setup_simulacion.sql
```

Descripción

Las funciones PL/pgSQL generan bots y actividad histórica con reglas de negocio: crecimiento irregular, picos de fin de semana, horarios frecuentes, segmentos de jugadores, abandono por mapa y probabilidad de compra.

Funciones

`admin_regenerate_bots(p_bot_count)`: elimina y recrea bots de prueba.

`simulate_player_matches(p_user_id, p_registered_at)`: genera partidas posteriores al registro.

`simulate_player_purchases(p_user_id, p_registered_at)`: genera compras posteriores al registro.

`admin_run_full_simulation`: coordina la simulación completa.

`admin_fill_missing_gameplay`: completa partidas faltantes.

`admin_fill_missing_purchases`: completa compras faltantes.

Clasificación

- Estructurada.
- Simulada.
- Histórica.
- Generada dentro de la base de datos.

Limitaciones

- Utiliza `random()` sin una semilla reproducible.
- No constituye todavía un pipeline ETL independiente.
- Debe documentarse la cantidad exacta de registros generados en cada ejecución.
- La simulación debe conservarse separada de los datos reales.

3. Supabase Realtime

Descripción

Canal de eventos basado en cambios de PostgreSQL. La aplicación escucha inserciones en `game_matches` para actualizar estadísticas del jugador y del leaderboard. También escucha compras del usuario autenticado para actualizar su historial de compras.

Eventos utilizados

Evento	Tabla	Consumidor	Acción
INSERT	<code>game_matches</code>	Dashboard del jugador	Recarga estadísticas y leaderboard
INSERT	<code>purchases</code>	Dashboard del jugador	Recarga historial y gasto personal

Controles

- El canal se elimina al desmontar el componente.
- Las compras se filtran por `user_id`.
- El usuario debe estar autenticado.
- La persistencia ocurre primero en PostgreSQL y después se notifica el evento.

Limitaciones

- El dashboard administrativo realiza carga inicial, pero no tiene la misma actualización en tiempo real.
- Deben documentarse desconexión, reintentos, eventos duplicados y mensajes al usuario para la bonificación completa.

4. Supabase Storage

Descripción

Almacena imágenes utilizadas por los productos de la tienda. El administrador carga archivos en el bucket configurado y la aplicación utiliza la URL pública para mostrar el producto.

Clasificación

- No estructurada.
- Archivo digital.
- Asociada al catálogo de productos.

Controles

- Normalización del nombre del archivo antes de subirlo.
- Uso de rutas dentro de la carpeta `productos/`.
- La URL se guarda en `products.image_url`.
- El bucket y sus políticas deben configurarse para no exponer archivos privados indebidamente.

Clasificación general

- Estructurados: tablas PostgreSQL y resultados de consultas.
- No estructurados: imágenes de productos.
- Históricos: perfiles, compras y partidas ya almacenadas.
- Datos en movimiento: eventos Realtime.
- Privados: correos, perfiles, compras y actividad individual.
- Simulados: bots, partidas y compras generadas por funciones SQL.
- Recolectados de una fuente real: registros generados por usuarios reales en la aplicación, cuando existan.

Reglas de privacidad

- No exportar correos electrónicos ni credenciales para el análisis.
- Anonimizar identificadores cuando los datos salgan de Supabase.
- Utilizar únicamente la información necesaria para responder preguntas de negocio.
- No mezclar bots con usuarios reales sin una columna o criterio documentado.
- Aplicar RLS en producción y comprobarlo con pruebas de acceso.

Parte III · Diccionario de datos

Propósito

Este diccionario describe las entidades y campos utilizados por TRIP para registrar usuarios, productos, compras, partidas, progreso y mejores tiempos. Los tipos indicados se basan en las consultas React y en las funciones SQL disponibles. Los tipos marcados como por confirmar deben verificarse directamente en el esquema de Supabase.

Convenciones

- Las fechas se almacenan como fecha-hora con zona horaria cuando se utiliza timestamptz.
- Los identificadores de usuarios y registros utilizan UUID cuando provienen de Supabase.
- Los importes monetarios se manejan como valores numéricos.
- Los datos de bots son simulados y se identifican por correos bot*@gmail.com.
- Las tablas deben aplicar las políticas RLS correspondientes.

profiles

Campo	Tipo	Descripción	Regla o validación	Ejemplo
id	UUID	Identificador del perfil y referencia al usuario autenticado	Obligatorio y único	uuid
username	Texto	Nombre público del jugador	Obligatorio y único	bot_25
role	Texto	Rol de acceso a la aplicación	Valores esperados: user, admin	user
created_at	Fecha-hora	Fecha de creación del perfil	No debe ser futura	2025-04-12T16:30:00Z

products

Campo	Tipo	Descripción	Regla o validación	Ejemplo
id	UUID o entero	Identificador del producto	Obligatorio y único	uuid
name	Texto	Nombre comercial del contenido	Obligatorio	Mesh principal
description	Texto	Descripción del producto	Puede ser nula	Contenido adicional
type	Texto	Tipo de contenido digital	Usa mesh, juego, asset y otro	mesh
price	Decimal	Precio actual del producto	Mayor o igual a cero	99.00
image_url	Texto/URL	Imagen pública del producto	Puede ser nula; debe ser una URL válida	https://...
image_position	Texto	Posición focal de la imagen	Puede ser una posición CSS	50% 50%

Campo	Tipo	Descripción	Regla o validación	Ejemplo
created_at	Fecha-hora	Fecha de alta del producto	No debe ser futura	2025-04-12T16:30:00Z
created_by	UUID	Administrador que creó el producto	Debe referenciar un usuario autorizado	uuid

purchases

Campo	Tipo	Descripción	Regla o validación	Ejemplo
id	UUID o entero	Identificador de la compra	Obligatorio y único	uuid
user_id	UUID	Jugador que realizó la compra	Debe existir en perfiles o auth.users	uuid
product_id	UUID o entero	Producto comprado	Debe existir en products	uuid
price_paid	Decimal	Precio pagado en la transacción	Mayor o igual a cero	99.00
created_at	Fecha-hora	Fecha y hora de la compra	No anterior al registro del usuario	2025-04-12T16:30:00Z

game_matches

Campo	Tipo	Descripción	Regla o validación	Ejemplo
user_id	UUID	Jugador que realizó la partida	Debe existir en perfiles	uuid
played_at	Fecha-hora	Momento de la partida	Igual o posterior al registro; no futura	2025-04-12T16:30:00Z
duration_seconds	Entero	Duración de la partida en segundos	Nulo si no finalizó; si existe, ≥ 0	245
death_type	Texto	Tipo de muerte, si ocurrió	enemigos, sobredosis, abstinencia, colapso o caídas; puede ser nulo	enemigos
consumptions	Entero	Número de consumos realizados en la partida	Mayor o igual a cero	3
map_name	Texto	Nombre del mapa jugado	Debe pertenecer al catálogo de mapas	Hospital
map_number	Entero	Número de orden del mapa	Entre 1 y el total de mapas	3
max_health_end	Entero	Vida al terminar la partida	Rango esperado de 0 a 100	72
damage_received	Entero	Daño recibido durante la partida	Mayor o igual a cero	28

progress

Campo	Tipo	Descripción	Regla o validación	Ejemplo
user_id	UUID	Jugador cuyo avance se registra	Debe existir en perfiles	uuid
Campos adicionales	Por confirmar	La aplicación y el SQL solo confirman la existencia de la tabla	Documentar al consultar el esquema real	-

leaderboard_best_times

La aplicación consulta esta vista o tabla con `select(*)` y utiliza los siguientes campos:

Campo	Tipo	Descripción	Regla o validación	Ejemplo
id	UUID o entero	Identificador del registro	Único	uuid
user_id	UUID	Jugador del récord	Debe existir en perfiles	uuid
username	Texto	Nombre mostrado en el ranking	Debe respetar la política de privacidad	jugador_25
map_name	Texto	Mapa del récord	Debe existir en el catálogo de mapas	Refugio
duration_seconds	Entero	Mejor tiempo en segundos	Mayor que cero	180
played_at	Fecha-hora	Fecha del récord	No futura	2025-04-12T16:30:00Z

Catálogos y dominios

Tipos de producto

mesh: producto repetible según la lógica de la tienda.

juego: compra única.

asset: compra única.

otro: compra única.

Mapas simulados

Calle Vacía, Habitación, Hospital, Callejón, Refugio y Recuerdo.

Tipos de muerte simulados

enemigos, sobredosis, abstinencia, colapso y caídas.

Reglas de integridad

1. Cada perfil debe corresponder a un usuario autenticado.
2. Cada compra debe corresponder a un usuario y un producto existentes.

3. Una compra no puede ocurrir antes del registro del usuario.
4. Cada partida debe corresponder a un jugador existente.
5. Las partidas no pueden estar en el futuro.
6. Los precios y duraciones no deben ser negativos.
7. Los administradores se excluyen de las métricas generales del dashboard.
8. El jugador solo debe consultar sus propias compras y partidas.
9. Los bots deben distinguirse de los usuarios reales y no deben alterar sus registros.

Pendientes de validación

- Confirmar tipos exactos, claves primarias y foráneas en Supabase.
- Documentar todos los campos de progress.
- Documentar campos exactos de leaderboard_best_times.
- Registrar nulos, rangos y cardinalidades observadas en el dataset final.
- Crear una versión exportable del diccionario junto con el dataset procesado.